Documento de Sustentación — Editor de Texto CLI integrado a eafitOS

Autor: Juan Fernando Romero
Asignatura: Sistemas Operativos — SO2026B
Proyecto: Editor de texto CLI (System Calls / POSIX)
Tamaño de equipo: 1 (individual) → alcance: comandos base (sección 4 del enunciado)

1. Qué se construyó

Un editor de texto interactivo (`edit`), integrado como un nuevo comando del shell educativo `eafitOS`. Al ejecutarlo, se entra a un sub-modo con su propio prompt (`edit>`) que soporta 5 comandos:

Comando	Función	Syscalls usadas
`o <archivo>`	Abre/crea el archivo	`open()` con `O_RDWR | O_CREAT`, modo `0644`
`p [n]`	Imprime todo el archivo, o la línea `n`	`lseek()`, `read()`, `write()` (FD 1)
`a <texto>`	Añade una línea al final	`lseek()` (`SEEK_END`), `read()`, `write()`
`d <n>`	Borra la línea `n`	`lseek()`, `read()`, `write()`, `ftruncate()`
`q`	Cierra el archivo y vuelve al shell	`close()`

Toda la manipulación del archivo editado usa exclusivamente syscalls POSIX de bajo nivel (`open/read/write/lseek/ftruncate/close`), nunca `fopen/fread/fwrite/fclose`, cumpliendo la restricción de I/O del enunciado (secciones 2.2 y 3). La interacción con el usuario (prompts, mensajes) sí usa `printf` / `fgets`, como permite el enunciado.

Archivos entregados: el código del shell con el editor integrado (`main.c`, `shell.h`, `cat_*.c`, el nuevo `editor.c`), el `Makefile` (targets `all` y `clean`) y el script de pruebas `demo_test.sh`.

2. Decisión de integración con el shell

Categoría nueva `editor`, no extensión de `datos`.

Los comandos de `datos` (`d_create`, `d_read`, `d_info`, `d_copy`) son operaciones de un solo disparo: reciben argumentos, ejecutan una acción y retornan de inmediato al prompt `eafitOS>`. El editor, en cambio, abre un **sub-modo interactivo propio** con su propio ciclo `open → prompt →`

`comando → ... → close`, cualitativamente distinto: el usuario permanece "dentro" del editor ejecutando múltiples comandos (`o`, `p`, `a`, `d`) hasta escribir `q`. Forzarlo dentro de `datos` habría escondido esta diferencia de comportamiento (comando atómico vs. sesión interactiva) y habría sido confuso de documentar en el `help`.

La integración consistió en: registrar una entrada nueva en la tabla `commands[]` de `main.c` (nombre `edit`, categoría `editor`, uso, descripción, syscalls y el handler `cmd_edit`), declarar el prototipo en `shell.h`, agregar la categoría al `help`, y añadir `editor.c` al `Makefile`. Los archivos `cat_*.c` existentes no se modificaron.

Ejecución in-process, sin `fork` / `execvp`.

`cmd_edit()` se registra igual que cualquier otro handler de la tabla `commands[]` y se ejecuta en el mismo proceso del shell, compartiendo la misma tabla de descriptores de archivo. El control se cede al editor cuando el bucle `main()` del shell llama al handler; el editor corre su propio bucle de lectura de comandos, y cuando el usuario escribe `q` (o envía fin de entrada con Ctrl+D) la función hace `close()` del archivo y retorna (`return 0`), devolviendo el control al bucle `main()` del shell — igual que retorna cualquier otro comando.

No se usó `fork` / `execvp` (como sí hace `p_exec` para lanzar binarios externos de Linux) porque el editor **no es un programa externo**: es lógica del mismo shell, no necesita aislarse en otro proceso, y evitar el `fork` simplifica el código (no hay que sincronizar con `waitpid`, ni preocuparse por procesos huérfanos si el editor falla).

3. Estructuras de datos

Se usan dos variables `static` a nivel de archivo en `editor.c`:

```
static int g_fd = -1;              // descriptor del archivo abierto (-1 = ninguno)
static char g_filename[PATH_MAX];  // nombre del archivo abierto
```

Por qué no un `struct EditorBuffer`: para el equipo de 1 persona el alcance es fijo (solo comandos base, sin multi-buffer ni undo/redo), así que agregar una estructura con campos que nunca se usarían (índice de líneas cacheado, lista de buffers, etc.) habría sido complejidad innecesaria. Si el proyecto creciera a más integrantes, el primer paso de refactor sería envolver `g_fd` / `g_filename` en un `struct EditorBuffer` — el cambio es mecánico porque toda la lógica ya depende de "el archivo actualmente abierto" como unidad.

Para borrar una línea se usa un búfer dinámico temporal (`malloc` / `free`) del tamaño exacto de los bytes que hay después de la línea borrada; se libera en cuanto termina la operación.

4. Cómo se determinan los límites de línea

Para el sistema operativo un archivo es solo una secuencia de bytes; las "líneas" se definen por el carácter `\n`. Se eligió **escaneo bajo demanda**: la función `find_line()` recorre el archivo byte a byte con `read()` contando `\n` cada vez que se necesita una línea, en vez de cachear un índice de offsets en memoria. Si la última línea no termina en `\n`, igual se cuenta como línea.

Justificación: para el tamaño de archivos que maneja un editor educativo esto es más que suficiente en rendimiento, y evita el problema de mantener un índice sincronizado tras cada `a`/`d` (que sería una fuente extra de bugs). Es la opción más simple de explicar y de verificar como correcta.

Borrado de una línea (`d n`): se ubica el inicio y el fin de la línea con `find_line()`, se leen con `read()` todos los bytes posteriores a ella, se reescriben con `write()` a partir del offset donde empezaba la línea borrada y finalmente se corta el sobrante del final con `ftruncate()`.

Añadir una línea (`a texto`): se va al final con `lseek(SEEK_END)`. Si el archivo no termina en `\n` (por ejemplo, uno creado con `d_create` del shell), primero se escribe un `\n` para que el texto quede como una línea nueva y no pegado a la última; luego se escribe el texto y su `\n`.

5. Manejo de errores

Todas las syscalls (`open`, `read`, `write`, `lseek`, `ftruncate`, `close`) se invocan a través de wrappers `checked_*()` en `editor.c` que verifican el valor de retorno `−1` y llaman a `perror()`. Quien llama a cada wrapper revisa el resultado y, si hubo error, cancela la operación en curso. En particular, en `d`, si cualquier lectura o escritura falla se aborta **antes** de truncar, para no dañar el archivo. También se verifica que `malloc()` no devuelva `NULL`.

Además, el editor valida en su propio bucle los usos inválidos antes de llegar a la syscall: línea inexistente o número inválido (`p 99`, `d 0`, `p abc`), comandos `p`/`a`/`d` sin haber abierto un archivo, comandos sin argumento (`a`, `d`, `o`) y comandos desconocidos. En todos los casos se muestra un mensaje claro y el editor sigue funcionando. Al salir (con `q` o con Ctrl+D) siempre se cierra el descriptor abierto, sin dejar fugas.

6. Cómo se probó

El script `demo_test.sh` compila el proyecto (`make clean` + `make all`) y envía por `stdin` una secuencia de comandos al shell que cubre:

- **Flujo normal:** crear un archivo con `edit`, agregar 3 líneas, imprimir todo, imprimir una línea, borrar una línea, volver a imprimir y salir; luego se verifica el contenido en disco con el

comando `d_read` que ya existía en el shell.

- **Casos borde y de error:** entrar a `edit` sin archivo y usar `p` / `a` (error controlado), abrir un archivo nuevo con `o`, imprimir un archivo vacío, pedir y borrar una línea que no existe.

Se ejecuta con: `./demo_test.sh` dentro de la carpeta del shell.